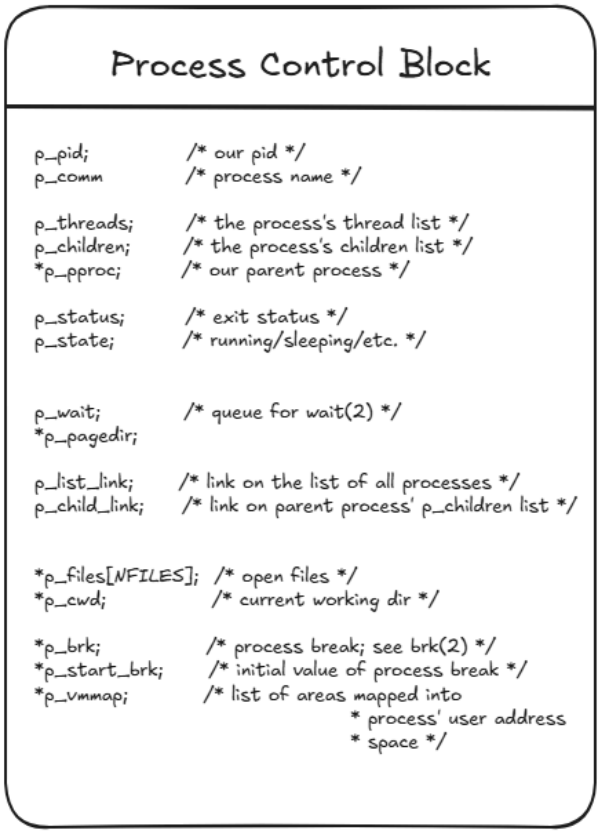
Part 1: Implementing **Processes, threads, and scheduler primitives**

Processes

Process Control block

Some process control block parameters:

1. **p_threads list**: List of all kernel threads belonging to this process.
2. **p_children list**: List of child processes created by this process.
3. **p_pproc:** Pointer to the parent process that created this process.
4. **p_state:** Indicates whether the process is running (PROC_RUNNING) or dead (PROC_DEAD).
5. **p_wait queue**: This is the queue a process puts itself on when it wants to wait on one of its children. When the child process terminates, it will broadcast on this queue, waking the waiting process.
6. **p_pagedir**: Pointer to the page directory defining the process's virtual memory mapping.
7. **p_list_link:** Link used to insert the process into the global list of all processes in the system.
8. **p_child_link**: Link used to insert the process into its parent's p_children list
9. **p_files**: Array storing all open file descriptors for the process.
10. **struct vnode *p_cwd:** Pointer to the current working directory vnode for filesystem operations.
11. **p_brk**: Pointer to the current end of the heap (program break) used for dynamic memory allocation.
12. **p_start_brk**: Pointer to the initial heap start address when the process was created.
13. **p_vmmap**: Data structure describing the virtual memory regions mapped into the process address space.



Parameters 9-13 will be discussed in detail in Parts 2 and 3.

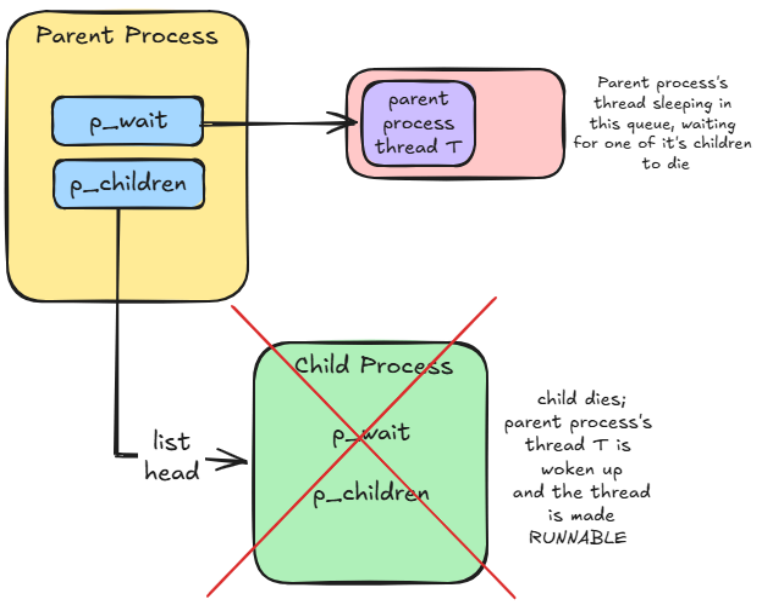
© 2026 Manasa BSV. All rights reserved.

Main Functions:

- **proc_create()**: Allocates and initializes a new process control block, assigns PID/parent/pagedir, links it into process lists, and marks it running.
- **proc_cleanup()**: Converts current process into a zombie by setting exit status and DEAD state, waking parent in p_wait queue, and reparenting children to init (or reaping if init).
- **do_waitpid()**: Waits for a matching child (or any child) to exit, returns its PID, writes exit status, and fully reaps child thread/process resources.
- **proc_kill()**: Kills a target process by calling do_exit if it is current, otherwise canceling its main thread with the given status. (calling kthread_cancel())
- **proc_thread_exited()**: Handles the exit of the last thread in a process by converting the thread's return value into the process exit status, calling proc_cleanup() to clean up process resources, and then yielding control so the scheduler can switch execution to another runnable thread.

Diagrams

1. p_wait queue

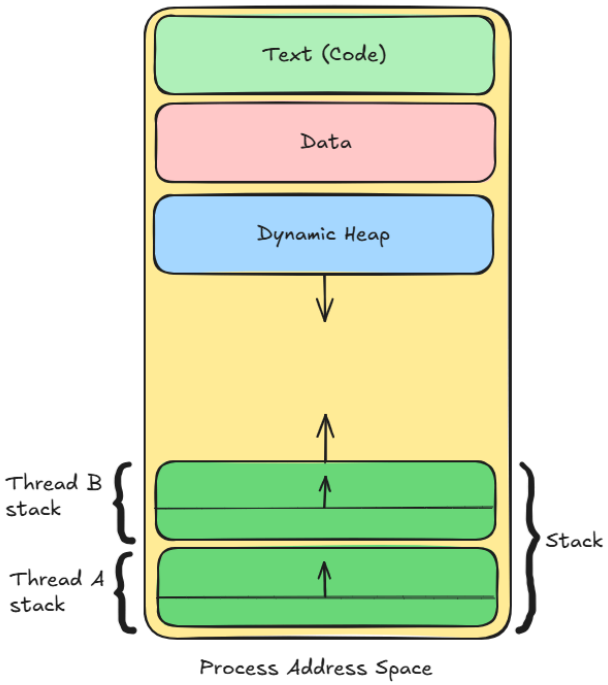


A. The parent process maintains a **p_wait** wait queue and a **p_children** list of its child processes.
B. When the parent calls **do_waitpid()**, its thread sleeps on p_wait while waiting for one of its children to exit.
C. When a child process dies, **proc_cleanup()** wakes the parent's thread on the parent's p_wait queue, making the thread's state "RUNNABLE" from "WAITING", so the parent can reap the child (inside proc_cleanup: sched_wakeup_on(&(current_proc->p_pproc->p_wait));)
D. When a child process finishes execution, proc_cleanup() marks it as PROC_DEAD, placing the process in a **zombie state** where its exit status is preserved until the parent reaps it.
E. In this state, the process has stopped executing but its process control block is not yet freed because the parent still needs to read its **exit status (return code)**.

© 2026 Manasa BSV. All rights reserved.

F. The child remains in the parent's p_children list while its exit status is stored in p_status.
G. When the parent wakes up and continues do_waitpid(), it retrieves the child's exit status and then reaps the child, freeing its remaining process structures.

2. Process Address Space



A. This diagram shows the process address space shared by multiple threads.
B. The text (code), data, and heap segments are shared by all threads, meaning every thread can access the same program code, global variables, and dynamically allocated memory.
C. However, each thread has its own stack, where function calls create stack frames containing local variables, parameters, and return addresses.
D. The stacks grow upward in the diagram toward the heap, illustrating that while threads share most memory regions, their stacks remain isolated to prevent interference between thread executions.

Threads

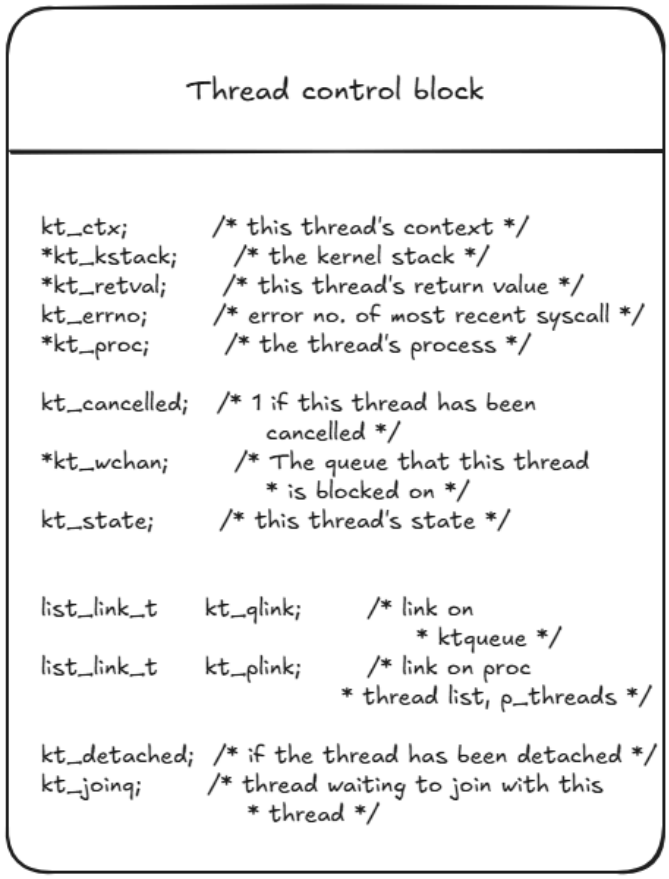
Thread control block

Some thread control block parameters:

1. **kt_ctx**: Stores the **CPU execution context** of the thread. This includes registers like the instruction pointer, stack pointer, etc. When the scheduler switches threads, it saves the current thread's context here and restores another thread's context.
2. **kt_kstack**: Pointer to the **kernel stack** allocated for this thread. Every kernel thread needs its own stack because function calls and local variables are stored here while the thread executes in kernel mode.

© 2026 Manasa BSV. All rights reserved.

3. **kt_cancelled**: Flag indicating whether the thread has been **cancelled**. Value is 1 if cancelled. Used when another thread calls kthread_cancel()
4. **kt_wchan**: Pointer to the wait queue that the thread is sleeping on. It might be waiting for I/O, waiting for a lock, or waiting in do_waitpid()
5. **kt_qlink**: Link used when the thread is inside a **queue** (ktqueue). Example queues: run queue, wait queue
6. **kt_plink:** Link used when the thread is in the **process's thread list** (p_threads).
7. **kt_state**: Stores the current state of the thread.



<u>Thread States</u> (k_state):

```
KT_NO_STATE,          /* illegal state */
KT_RUN,               /* currently running, or on runq */
KT_SLEEP,             /* blocked for an indefinite amount of time */
KT_SLEEP_CANCELLABLE, /* blocked, but this sleep can be cancelled */
KT_EXITED             /* has exited, waiting to be joined */
```

<u>Main functions</u>

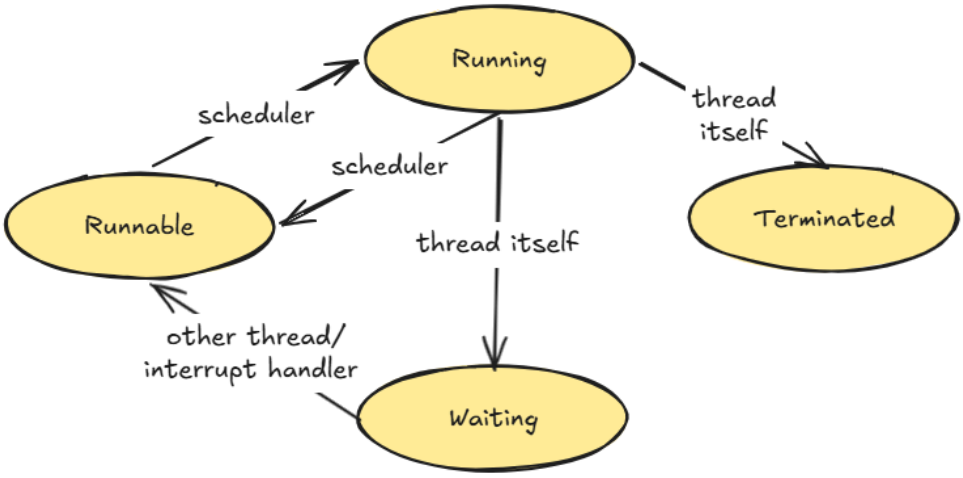
- **kthread_create()**: Allocates a kernel thread and stack, initializes context/state for a target process, and inserts it into that process's thread list.

© 2026 Manasa BSV. All rights reserved.

- **kthread_cancel()**: If target is current thread it exits immediately; otherwise it records cancellation/retval and lets scheduler cancellation logic handle wakeup.
- **kthread_exit()**: Stores thread return value, marks current thread exited after invariant checks, and transfers control to process-exit handling.

Diagrams

1. Thread life cycle



A. A thread starts in “RUNNABLE” state and waits in the run queue until the scheduler selects it to run
B. The scheduler switches a thread’s state from “RUNNABLE” to “RUNNING”. The scheduler decides who to run next inside the CPU
C. A thread goes from running to waiting when a blocking call is made by the thread itself (scheduler is not involved here)
D. The scheduler switches a thread’s state from “RUNNING” to “RUNNABLE” when the thread uses up its execution quantum

Scheduler

Main functions

- **sched_wakeup_on()**: Removes one sleeping thread from a wait queue (like a process’s wait queue, OR mutex queue), calls sched_make_runnable() to enqueue it onto the run queue.
- **sched_make_runnable()**: Raises IPL, marks a thread runnable, pushes it to run queue safely, then restores previous IPL.
- **sched_cancellable_sleep_on():** Checks if the current thread is already cancelled (returns -EINTR if so); otherwise marks it KT_SLEEP_CANCELLABLE, puts it on the wait queue, performs a context switch, and returns -EINTR if cancelled while sleeping or 0 if it wakes normally.
- **sched_cancel()**: Marks a thread as cancelled (kt_cancelled = 1) and, if it is in cancellable sleep, removes it from the wait queue and makes it runnable.

© 2026 Manasa BSV. All rights reserved.

- **sched_switch()**: Protects the run queue by raising IPL, selects the next runnable thread, switches the CPU context to it, and restores the previous IPL after the switch.

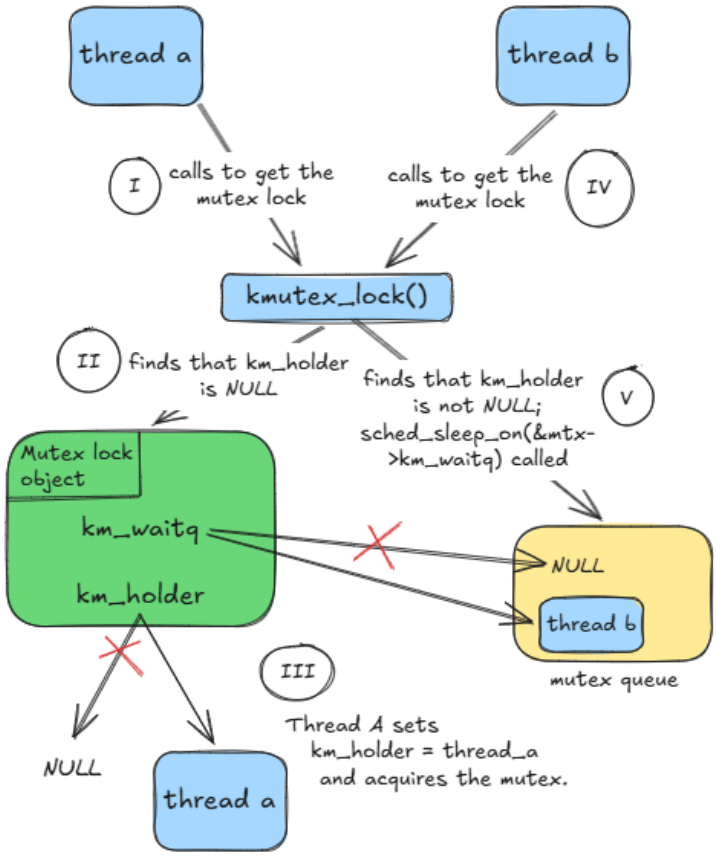
Mutex Locks

Implement kernel mutex synchronization primitives

- **kmutex_lock()**: Acquires mutex if free, otherwise blocks current thread on mutex wait queue until wakeup.
- **kmutex_lock_cancellable()**: Same as lock but uses cancellable sleep and returns interruption status when cancellation occurs.
- **kmutex_unlock()**: Releases mutex ownership and, if waiters exist, wakes one waiter and transfers lock ownership to that thread.

Conceptual Diagrams

1. Mutex lock

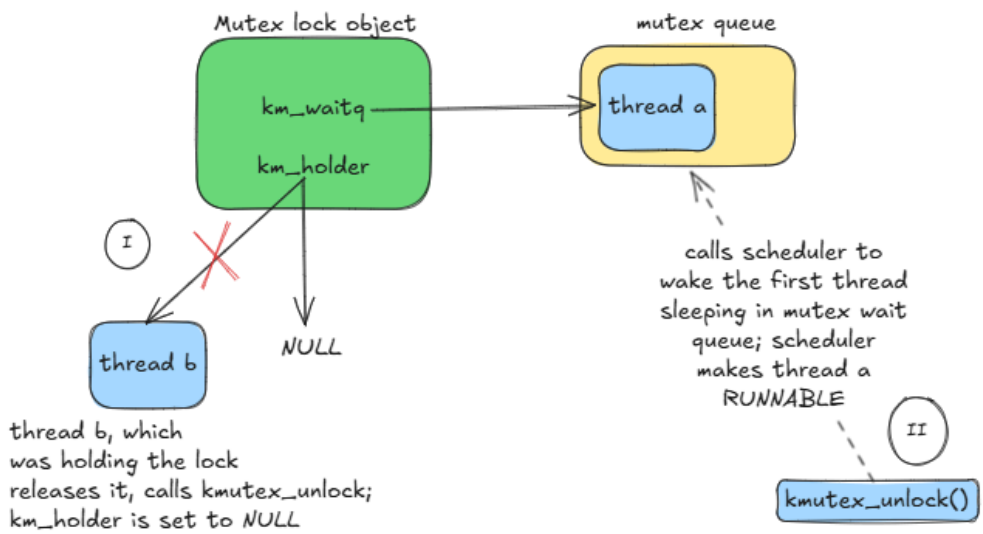


A. Thread **A** first calls kmutex_lock() to acquire the mutex. Since km_holder is **NULL**, the mutex is free, so thread A sets km_holder = thread_a and acquires the lock.
B. Later, thread **B** also calls kmutex_lock(), but it finds that km_holder is **not NULL** because thread A is holding the mutex.

© 2026 Manasa BSV. All rights reserved.

C. As a result, sched_sleep_on(&mtx->km_waitq) is called, placing thread B into the mutex wait queue (km_waitq) and blocking it.
D. Thread B will remain asleep in the wait queue until the mutex is released and the scheduler wakes it.

2. Mutex unlock



A. Thread B, which holds the mutex, calls kmutex_unlock() to release the lock.
B. First, km_holder is set to NULL, and then kmutex_unlock() calls sched_wakeup_on() to wake the first thread sleeping in the mutex wait queue, and the scheduler makes it "RUNNABLE" (adds it to the run queue).
C. Thread A is moved to the run queue and will later be scheduled to run, where it resumes kmutex_lock() and may acquire the mutex.

© 2026 Manasa BSV. All rights reserved.